DriftLab

A plain-English guide to what it does and how to run it

This guide explains DriftLab without assuming you write software for a living. It covers the problem it solves, how it works, what its results mean, how to run it on your own machine, and what every important word means. The technical companion documents (README, RUNBOOK, CERTIFICATION-GUIDE) live in the same folder.

1. The problem, in one minute

Companies increasingly use **AI agents** — software that doesn't just answer questions but makes decisions and takes actions on its own: approving a loan, routing a customer complaint, moving a case forward. Before such an agent goes live, someone writes down the rules it must follow: *loans above 50,000 need a human reviewer; it may only use these five tools; after an error it must escalate to a person.*

The problem is what happens **after** launch. Agents change quietly — a model gets upgraded, a prompt gets tweaked, a new tool gets added — and their behavior drifts away from the agreed rules without anyone noticing. There's no crash and no alarm. Weeks later, an auditor or regulator asks: *“Can you prove your AI followed your rules last quarter?”* — and today, almost nobody can. Monitoring tools show *what the agent did* (useful for debugging), but nothing checks it against *what it was allowed to do*.

2. What DriftLab does

DriftLab is a **compliance observer for running AI agents**. Think of it as three things in one: a **security camera** that records every decision the agent makes, an **inspector** that regularly compares those recordings against the certified rulebook, and a **small laboratory** that double-checks every suspected violation before anyone gets alarmed. At the end of each check it prints the paperwork an auditor would ask for.

The whole cycle, step by step:

	STEP	WHAT HAPPENS
1	**Record**	The agent keeps a diary. Every decision — which tool it used, where it sent a case, whether it asked a human — is written down as one line in a locally stored log file.
2	**Compare**	On a schedule, DriftLab reads the diary and compares it, rule by rule, against the certified rulebook. This step is pure arithmetic — no AI judgment involved. The same diary and the same rulebook always give exactly the same answer.
3	**Double-check**	For every suspected violation, DriftLab forms a precise claim (“high-value cases are skipping human review”) and re-tests it against a replay of the recordings. Only claims that reproduce are labelled CONFIRMED.

	STEP	WHAT HAPPENS
4	**Report**	Each check produces a compliance score (0–100), a findings list with the evidence attached, and structured evidence artifacts. A dashboard shows the whole fleet at a glance; an optional email digest arrives only when something actually needs attention.

3. The rulebook: seven kinds of rules

The rulebook (DriftLab calls it a **certified baseline**) is a file your compliance team signs off. It answers seven questions about the agent. You don't have to answer all seven on day one — anything you leave out simply isn't monitored yet.

RULE FAMILY	PLAIN-ENGLISH MEANING
Tools	Which tools may the agent use, and where? Like a staff badge that only opens certain doors.
Routes	Which paths may a case take through the agent? A loan may go from “assess” to “human review”, but never straight from “intake” to “payout”.
Human review	Which decisions must a person see before they count? E.g. any loan above 50,000, or any angry customer.
After errors	When something fails, what must happen next? Usually: hand the case to a human, don't just carry on.
Points of no return	Which actions can't be undone (paying out money, sending a customer a commitment)? Any rule broken on the way to one of these is treated as far more serious.
Autonomy budget	What share of decisions may the agent take fully on its own? E.g. at most 40% — the rest must involve a person somewhere.
Mandatory steps	Which steps must happen in every single case (e.g. the risk assessment), and the duty to keep recording the diary at all.

Technically, the rulebook is stored in **OSCAL** — a machine-readable compliance format published by NIST (the US standards institute). DriftLab v0.1 produces OSCAL 1.1.2-shaped evidence artifacts, with one documented assessment-plan deviation while per-agent system security plans are still on the roadmap. Treat them as structured inputs to SOC 2, ISO/IEC 42001 and EU AI Act evidence workflows, not yet as proof of strict OSCAL conformance.

4. What a result looks like

DriftLab ships with a built-in demonstration: a fictional **loan-officer** agent whose recordings contain four planted rule violations, and a well-behaved **support-triage** agent as a control. Running the demo takes a few seconds and finds exactly this:

FINDING	WHAT HAPPENED	SEVERITY	DOUBLE-CHECK VERDICT
Human review skipped	5 of the 8 high-value loans went through without a person seeing them.	CRITICAL — the skipped review sits on the path to paying out money	CONFIRMED (reproduced on 5 of 8 matching cases)
Too much autonomy	65% of decisions were fully automatic; the certified budget is 40%.	HIGH	CONFIRMED (the excess holds in every time window — it's sustained, not a blip)
Unapproved tool	The agent started using a data source that was never certified.	HIGH	CONFIRMED (2 uses reproduced)
Error handled wrongly	After a failure, the agent carried on instead of escalating to a person.	HIGH	CONFIRMED (1 of 2 errors mishandled)

The drifted agent scores **5 / 100**. Its well-behaved twin scores **100 / 100** with zero findings — DriftLab does not invent problems where there are none. Severity levels read as follows: **CRITICAL** means a certified rule was broken on a path that reaches a point of no return — treat it like an incident. **HIGH** means a rule was clearly broken. **MEDIUM/LOW** are degradations and data-quality warnings. The score starts at 100 and loses points per finding, weighted by severity.

The three double-check verdicts

CONFIRMED — the violation reproduced when re-tested against the recordings; the report includes the exact cases and counts. Escalate with confidence. **REFUTED** — the claim did not reproduce. Either the problem was fixed (re-testing after a fix is free), or a false alarm was caught before it wasted anyone's time. **INCONCLUSIVE** — not enough matching cases to say either way; collect more recordings first. DriftLab deliberately refuses to cry wolf on thin evidence.

5. How to run it

You need a Mac or Linux machine with Python 3.11 or newer and the **uv** tool. The core runs locally with no account or cloud service required. If you enable the optional OMLX analyst, trace-derived context is sent only to the operator-controlled OMLX endpoint you configure.

First contact (five minutes)

```
git clone https://github.com/sw30labs/driftlab.git
cd driftlab
uv sync --all-extras    # installs everything into a local environment
uv run driftlab demo    # runs the built-in demonstration above
```

```
uv run driftlab dashboard  # then open http://127.0.0.1:8321 in a browser
```

The dashboard shows every monitored agent with its score, its findings, its double-check verdicts, and download links for the evidence artifacts. Green means compliant; red means look now.

Watching your own agent (three steps)

A. Record the diary	Have your agent write its decisions to a log file (one JSON line per decision). If it's built with LangGraph/LangChain, DriftLab ships a ready-made recorder you attach with two lines of code. Otherwise any process that emits the simple diary format works.
B. Write the rulebook	Copy the demo rulebook and adapt it: your tools, your routes, your review thresholds. The CERTIFICATION-GUIDE walks through this line by line. Typos fail loudly on purpose — a misspelled rule must never silently certify less than you meant.
C. Register the agent	Add one entry (name, priority 1–7, rulebook file, diary file) to a small list called the registry, and point DriftLab at it. Priority 1 means “check daily”; 7 means “monthly”.

Making it automatic

One command — `driftlab monitor run --scheduled` — checks whatever is due and goes back to sleep. Put it on a daily timer (ready-made macOS and cron templates are in the `ops/` folder) and DriftLab decides internally which agents actually need checking, based on their priority and how long ago they were last seen. Running it twice does no harm. If you configure email, a digest is sent *only* when there is something worth reading: a new critical finding, a confirmed drift, a score drop, or an error. A broken mail server never stops the checks themselves.

What each check leaves behind

Four files per agent per check, saved locally: **report.md** — the human-readable summary; **ledger.json** — the findings with evidence, written so that re-running the same check produces a byte-for-byte identical file (an auditor can verify it themselves); and **assessment-plan.json / assessment-results.json** — OSCAL 1.1.2-shaped evidence artifacts, including positive attestations for every rule that was checked and held. v0.1 documents one non-strict assessment-plan reference until per-agent SSP output lands.

6. What DriftLab will never do

It never touches your live agent. All double-checks run against replays of the recordings, in memory. The code physically refuses to probe anything that isn't an isolated copy, and the test suite proves the entire double-check machinery runs with networking disabled.

The core path needs no external service. Diaries, rulebooks, findings, and reports stay on operator-controlled systems, and the dashboard works with no internet connection. If the optional OMLX analyst is enabled, trace-derived context goes only to the operator-configured OMLX endpoint — not to a third-party OpenAI service.

AI never decides a finding. DriftLab can optionally use an operator-controlled OMLX model to add readable summaries to reports — prose only. Every finding, severity, score, and verdict comes from the deterministic comparison and the replayed evidence. Unplug the language model and every number stays identical (this is tested, not promised).

It never buries a decision you made. If you formally accept a known issue (a “waiver”), the finding stays visible on every report, marked as waived, with an expiry date — acceptance lowers the alarm level, never the visibility.

7. Command cheat-sheet

COMMAND	WHAT IT DOES
`driftlab demo`	Run the built-in demonstration end to end
`driftlab agents`	List monitored agents and when each is next due
`driftlab check --agent NAME`	Check one agent right now
`driftlab monitor run`	Check whatever is due (add --force to check everything)
`driftlab monitor status`	History of recent monitoring runs
`driftlab report CYCLE-ID`	Print the human-readable report of one check
`driftlab dashboard`	Serve the web dashboard on http://127.0.0.1:8321

Prefix with `uv run` when working inside the project folder.

8. Glossary

Agent	Software that makes decisions and takes actions on its own, usually powered by an AI model.
Trace / diary	The chronological record of every decision an agent made, one line per decision.
Baseline / rulebook	The certified, machine-readable statement of how the agent is allowed to behave.
Drift	The gap between what the agent was certified to do and what it actually does, appearing gradually and silently.

Human-in-the-loop (HITL)	A rule that certain decisions must be reviewed by a person before they take effect.
Critical sink / point of no return	A step whose effects can't be taken back — paying out funds, sending a binding reply.
Micro-experiment / probe	A controlled re-test of one suspected violation against a replay of the recordings; produces CONFIRMED, REFUTED, or INCONCLUSIVE.
Sandbox / replay	The isolated, in-memory copy of the recordings that experiments run against — never the live system.
Deterministic	Same inputs, same outputs, every single time. No randomness, no AI judgment. The property that makes results auditable.
Compliance score	0–100 summary per check: starts at 100, loses points per finding, weighted by severity.
OSCAL	The NIST-published machine-readable format for compliance documents. DriftLab v0.1 follows its structures but is not yet strictly schema-validated.
Waiver	A formally accepted, time-boxed exception to a finding — visible on every report until it expires.

DriftLab v0.1 · sw30labs · Nicolas Cravino · ncravino@mac.com — Companion documents: README.md (technical overview), docs/RUNBOOK.md (day-to-day operations), docs/CERTIFICATION-GUIDE.md (writing a rulebook), ROADMAP.md (what's next).